

Tutorial 6

SC1006, CZ1106

Polling, Interrupts and DMA

1) Consider the system diagram in Figure 1 of case study notes.

- The processor UART peripheral in its Serial I/O module is configured to receive data with format of 1 start-bit, 7 data-bits, 1 parity-bit and 1 stop-bit.
- Each time the UART peripheral receives a character, it'll store the data into a buffer.
- It will then interrupt the CPU to notify CPU that there is data available.
- The CPU will then execute the Interrupt Service Routine (ISR) to read the character received.
- Interrupt Latency is the term used to describe the time between interrupt request and entrance to ISR.
- The minimum interrupt latency for the CPU is 10 μ s and it takes 90 μ s for the CPU to execute the instructions in the ISR (read the received data from the buffer).
- Assume that the UART data is transferred back to back i.e. no delays between each UART packets.

10 bits

What is the maximum baud rate that can be supported with this UART interface?

Sequence of event:

- External device send a character via UART to processor.
- The processor UART peripheral receives the character in its buffer and interrupts the CPU
- CPU proceed to execute the ISR corresponding to the UART interrupt to read and transfer the character to main memory.
- Total time required to transfer one character = Interrupt Latency + ISR execution.

$$1 \text{ micro sec} = 10^{-6} \text{ sec} \quad 10+90=100 \text{ micro sec} = 0.0001 \text{ sec} \quad 1/0.0001=10000$$

$$1/(10+90) \mu\text{s} = 10000 \text{ ISRs serviced per second}$$

Each character transfer involved 10 bits on UART (S-7D-P-STP).

$$10 \text{ bits} * 10000 = 100 \text{ Kbits per second}$$

2) The specifications of the system in Figure 1 (case study notes) requires the buttons to be asserted 400 times over a 24-hour period. Given that the processor consumes 0.1mA current when it is idling and 50mA when it is in active mode i.e. running code, answer the following.

(a) Supposed polled IO technique is used to sample the button status. Given that the processor poll the buttons every 100ms, each poll requires the processor to be active for 10ms, what is the average current consumed over the 24-hour period?

Polled (or programmed) I/O: **The CPU periodically manually checks if there are any I/O requests available**. If there isn't, it keeps executing its normal workflow. If there is, it handles the I/O request instead.

(a) Polled every 100ms, each poll takes 10ms

⇒ 90ms(idle), 10ms(active).

⇒ Average current = $0.9 \times 0.1\text{mA} + 0.1 \times 50\text{mA} = 5.09\text{mA}$

2) The specifications of the system in Figure 1 (case study notes) requires the buttons to be asserted 400 times over a 24-hour period. Given that the processor consumes 0.1mA current when it is idling and 50mA when it is in active mode i.e. running code, answer the following.

(b) If interrupt-driven I/O is used to sample the button status, what is the average current consumed in the same 24-hour period? Given that it takes 20ms to service each interrupt, including interrupt latency and ISR execution.

Interrupt-Driven I/O: The CPU doesn't need to manually check for I/O requests.

(b) Interrupt mechanism.

⇒ Interrupted 400 times over 24-hour

⇒ Active time = $400 * 20\text{ms} = 8000\text{ms} = 8 \text{ sec}$

⇒ Average current consumed over 24-hour

$$= [8 * 50\text{mA} + ((24 * 60 * 60) - 8) * 0.1\text{mA}] / [24 * 60 * 60]$$

$$= 0.1046\text{mA}$$

Seconds in 24 hours

Seconds in 24 hours

- 2) The specifications of the system in Figure 1 (case study notes) requires the buttons to be asserted 400 times over a 24-hour period. Given that the processor consumes 0.1mA current when it is idling and 50mA when it is in active mode i.e. running code, answer the following.
- (c) Given that a battery with capacity of 2000mAh means that it can supply 2000mA continuously for 1 hour, or 1000mA continuously for 2 hours. How long would a battery of 2000mAh last when used in (a) and (b) above?

(c) Use case (a) (Polling)

- a. Average current consumed 5.09mA $2000/5.09/24=16.3719711853$
- b. Battery life = $2000\text{mAh}/5.09\text{mA} = 393 \text{ hours} = 16.4 \text{ days}$

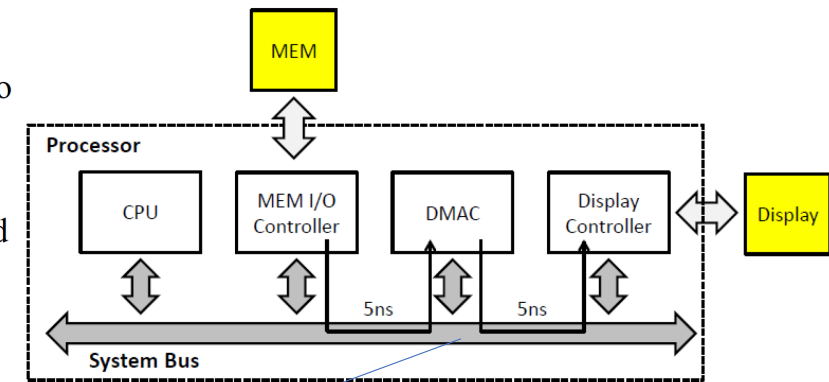
Use case (b) (Interrupt)

- a. Average current consumed 0.1046mA $2000/0.1046/24/365=2.18270078665$
- b. Battery life = $2000\text{mAh}/0.1046\text{mA} = 19120 \text{ hours} = 796 \text{ days} = 2.18 \text{ years}$

- 3) Given that DMAC and CPU share one system bus, list four possible factors that might affect the transfer rate of a DMAC and explain how they affect the DMAC transfer rate.
- CPU competes with the DMAC (DMA Controller) where system bus utilization is concerned. Higher CPU utilization rate (of the system bus) means lesser chance for DMAC to use the same bus, hence DMA data transfer rate will be affected.
 - The competition for resources is also affected by the priority of DMAC with respect to the CPU. Depending on processor design, priority of CPU with respect to DMAC can be fixed or configurable. The one with higher priority will have higher chance of hitting their maximum possible transfer rate.
 - Maximum Data transfer rate is limited by the slowest device in the whole data path consisting of the source device, DMAC and destination device. If there is a memory or I/O device that is slow, it'll affect the overall DMA transfer rate.
 - All things equal, the mode of transfer used by the DMAC (burst, cycle stealing or transparent) will affect the DMAC transfer rate.

4) Consider the system in Figure 1 (Case study notes). Given that

- The processor's system bus is capable of supporting simultaneous transfer of up to 3 bytes of data at one time.
- DMA is used to transfer video data from Memory to Display Controller module.
- Each video pixel data consist of three bytes (Red, Green, Blue) and are transferred simultaneously on the system bus on each bus cycle.
- Each transfer on the system bus takes 5ns and transfer of the bus control between the CPU and the DMAC takes 100ns.
- Assume that DMAC is using Fetch-and-Deposit DMA.
- Note that 1Kbyte = 1024 Byte.



- (a) Given that the video is output at a rate of 30 frames per second and each video frame has a resolution of 1920x1080 pixels. If burst-mode was used by the DMAC to burst one frame of video data at a time, would the DMAC be able to transfer each video frame completely?

Burst Mode -

- In this mode Burst of data (entire data or burst of block containing data) is transferred before CPU takes control of the buses back from DMAC.

Each transfer of the system bus will transfer 3 bytes of video data (RGB) and take $2 \times 5\text{ns}$.

Number of transfers in $1/30$ seconds $\therefore 1920 \times 1080 = 2073600$ transfers. Each pixel – 3 bytes

Total time taken

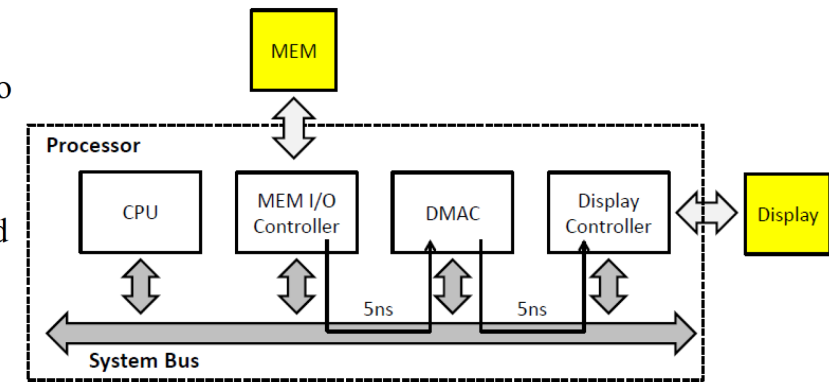
= Bus control request + actual transfer + Bus release

= $100\text{ns} + 2073600 \times 2 \times 5\text{ns} + 100\text{ns} = 0.0207362 \text{ sec} < 1/30\text{sec}$ $1/30 = 0.03333$

$\Rightarrow$ DMAC is able to transfer each frame of video before the start of the next frame request.

4) Consider the system in Figure 1 (Case study notes). Given that

- The processor's system bus is capable of supporting simultaneous transfer of up to 3 bytes of data at one time.
- DMA is used to transfer video data from Memory to Display Controller module.
- Each video pixel data consist of three bytes (Red, Green, Blue) and are transferred simultaneously on the system bus on each bus cycle.
- Each transfer on the system bus takes 5ns and transfer of the bus control between the CPU and the DMAC takes 100ns.
- Assume that DMAC is using Fetch-and-Deposit DMA.
- Note that 1Kbyte = 1024 Byte.



(b) Repeat the calculation if the DMAC is using cycle-stealing mode, assume that DMAC needs to wait at least 5 instructions before it could request for control of the system bus again.

Time taken for one transfer $5/400\text{MHz}=12.5\text{ns}$ $100 + 10 + 100 + 12.5 = 222.5 \text{ ns}$

= Bus control request + 1 transfer + Bus release + 5*CPU Clock cycles wait

= $100\text{ns} + 2*5\text{ns} + 100\text{ns} + 5*(1/400\text{Mhz}) = 222.5\text{ns}$

$1/30=0.03333$

Total time taken for 2073600 transfers = $2073600*222.5\text{ns} = 0.461376 \text{ sec} > 1/30\text{sec}$

⇒ DMAC will not be able to complete the transfer of one video frame before the next frame request comes in

DMA controllers can operate in a *cycle stealing* mode in which they take over the bus for each data to be transferred and then return control to the CPU.

Note: the '400 Mhz' for CPU clock is taken from the processor information in the case study notes.